Python Dictionary

What is a Dictionary?

- **A dictionary is an unordered**

  mutable collection of key-value pairs in Python.

- **Key Characteristics:**

  - Key-Value Pairs: Each element is a pair {key: value}
  - Mutable: Can add, remove, and modify key-value pairs
  - Keys must be unique and hashable (immutable types)
  - Values can be any type (mutable or immutable)
  - Insertion-ordered: Python 3.7+ preserves insertion order

- **Dictionaries are written with curly braces { }:**

  ```
  student = {'name': 'Alice', 'age': 20, 'dept': 'CS'}
  ```

- **Also called: hash maps, or mappings**

Creating Dictionaries

- **Empty Dictionary:**

```
d = {}              # Most common way
d = dict()          # Using constructor
```

- **With Key-Value Pairs:**

```
student = {'name': 'Alice', 'age': 20, 'marks': 85.5}
```

- **Using dict() with keyword arguments:**

```
d = dict(name='Alice', age=20, marks=85.5)
```

- **Fromkeys() — same value for all keys:**

```
d = dict.fromkeys(['a', 'b', 'c'], 0)
# → {'a': 0, 'b': 0, 'c': 0}
```

Accessing & Modifying Values

- **Accessing Values (by key):**

```
student = {'name': 'Alice', 'age': 20}
student['name']     →  'Alice'
student['age']      →  20
```

- **Using get() — safer access (no KeyError!):**

```
student.get('name')          →  'Alice'
student.get('grade')         →  None  (no error!)
student.get('grade', 'N/A') →  'N/A'  (default value)
```

- **Modifying / Adding Values:**

```
student['age'] = 21          # Modify existing key
student['grade'] = 'A'       # Add new key-value pair
```

Accessing & Modifying Values

- **Using setdefault() — set if key doesn't exist:**

```
student.setdefault('city', 'New York')
# Adds 'city': 'New York' only if 'city' not present
```

- **Using update() — merge another dictionary:**

```
student.update({'phone': '12345', 'age': 22})
# Modifies 'age' and adds 'phone'
```

Dictionary Operators

- **Membership Testing (keys only!):**

```
student = {'name': 'Alice', 'age': 20}

'name' in student       →  True
'grade' in student      →  False
'Alice' in student      →  False  (checks KEYS, not values!)

'name' not in student   →  False
'grade' not in student  →  True
```

Dictionary Operators

- **Equality Comparison:**

```
{'a': 1, 'b': 2} == {'b': 2, 'a': 1}  →  True
# Order doesn't matter for comparison!

{'a': 1, 'b': 2} == {'a': 1, 'b': 3}  →  False
```

Removing Elements from Dictionary

- **pop(key) — Removes key and returns its value:**

```
student = {'name': 'Alice', 'age': 20, 'dept': 'CS'}
age = student.pop('age')
# age → 20, student → {'name': 'Alice', 'dept': 'CS'}

student.pop('grade')        #    KeyError if key not found!
student.pop('grade', None)  #    Returns None if not found
```

- **popitem() — Removes and returns the LAST inserted item:**

```
item = student.popitem()
# item → ('dept', 'CS')  (as a tuple)
```

Removing Elements from Dictionary

- **del statement — Remove a specific key:**

```
del student['name']
# student → {'dept': 'CS'}

del student       # Deletes the entire dictionary!
```

- **clear() — Remove all items:**

```
student.clear()
# student → {}  (empty dictionary)
```

Dictionary Views: keys(), values(), items()

- **These methods return 'view objects' — dynamic reflections of the dict**

  Views are dynamic — they reflect dictionary changes

- **keys() — Returns all keys:**

```
student = {'name': 'Alice', 'age': 20}
student.keys()  →  dict_keys(['name', 'age'])
```

Example -

```
keys = student.keys()
student['grade'] = 'A'
print(keys)  # Now includes 'grade' too!
```

Dictionary Views: keys(), values(), items()

- **values() — Returns all values:**

```
student.values()  →  dict_values(['Alice', 20])
```

- **items() — Returns all key-value pairs as tuples:**

```
student.items()  →  dict_items([('name', 'Alice'), ('age', 20)])
```

- **Iterating over dictionaries:**

```
for key in student:           # Iterates over keys
```

Built-in Functions with Dictionaries

- **len(d) → Returns number of key-value pairs**

```
len({'a': 1, 'b': 2})  →  2
```

- **max(d) / min(d) → Returns max/min KEY (not value!)**

```
max({'b': 2, 'a': 1})  →  'b'   (string comparison)
min({3: 'x', 1: 'y'})  →  1
```

- **sum(d) → Returns sum of all KEYS (must be numeric)**

```
sum({1: 'a', 2: 'b', 3: 'c'})  →  6
```

- **sorted(d) → Returns sorted list of KEYS**

```
sorted({'c': 1, 'a': 2, 'b': 3})  →  ['a', 'b', 'c']
```

Built-in Functions with Dictionaries

- **dict(iterable) → Converts iterable to dictionary**

```
dict([('x', 10), ('y', 20)]) → {'x': 10, 'y': 20}
```

- **enumerate(d) → Returns index-key pairs**

```
list(enumerate({'a': 1, 'b': 2}))
→ [(0, 'a'), (1, 'b')]
```

- **all(d) / any(d) → Check truthiness of keys**

```
all({1: 'a', 2: 'b'}) → True
any({0: 'a', 1: 'b'}) → True
```

Dictionary Methods (Part 1)

- **get(key, default) — Safe access with default:**

```
d = {'a': 1, 'b': 2}
d.get('c', 0)  →  0
```

- **setdefault(key, default) — Set if key missing:**

```
d.setdefault('c', 3)  # d → {'a': 1, 'b': 2, 'c': 3}
d.setdefault('a', 99) # d unchanged, returns 1
```

- **update(other_dict) — Merge dictionaries:**

```
d1 = {'a': 1, 'b': 2}
d2 = {'b': 20, 'c': 3}
d1.update(d2)  # d1 → {'a': 1, 'b': 20, 'c': 3}
```

Dictionary Methods (Part 1)

- **copy() — Shallow copy of the dictionary:**

```
d2 = d1.copy()
```

- **fromkeys(keys, value) — Create dict with same value:**

```
dict.fromkeys(['a', 'b', 'c'], 0)
# → {'a': 0, 'b': 0, 'c': 0}
```

Dictionary Methods (Part 2)

- **pop(key, default) — Remove and return value:**

```
d = {'a': 1, 'b': 2, 'c': 3}
d.pop('b')         →  2,  d → {'a': 1, 'c': 3}
d.pop('z', None)   →  None  (no error!)
```

- **popitem() — Remove and return last item:**

```
d.popitem()  →  ('c', 3)  (LIFO order, Python 3.7+)
```

- **clear() — Remove all items:**

```
d.clear()  # d → {}
```

- **keys() — View of all keys**
- **values() — View of all values**
- **items() — View of all (key, value) pairs**

Dictionary Comprehensions

- **A concise way to create dictionaries from iterables:**

  Syntax: **{key_expr: value_expr for item in iterable if condition}**

- **Examples:**

```
# Squares of numbers 0 to 4
{x: x**2 for x in range(5)}
→  {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Word lengths
words = ['apple', 'banana', 'cherry']
{word: len(word) for word in words}
→  {'apple': 5, 'banana': 6, 'cherry': 6}
```

Dictionary Comprehensions

```
# Filtered comprehension (even numbers only)
{x: x**2 for x in range(10) if x % 2 == 0}
→  {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}

# Inverting a dictionary
d = {'a': 1, 'b': 2, 'c': 3}
{v: k for k, v in d.items()}
→  {1: 'a', 2: 'b', 3: 'c'}
```

Nested Dictionaries

- **Dictionaries can contain other dictionaries as values:**

```
students = {
    'Alice': {'age': 20, 'dept': 'CS', 'marks': 85},
    'Bob': {'age': 21, 'dept': 'IT', 'marks': 78},
    'Charlie': {'age': 20, 'dept': 'CS', 'marks': 92}
}
```

- **Accessing nested values:**

```
students['Alice']['age']     →  20
students['Bob']['dept']      →  'IT'
students['Charlie']['marks'] →  92
```

Nested Dictionaries

- **Modifying nested values:**

```
students['Alice']['marks'] = 90
```

- **Adding nested entries:**

```
students['Dave'] = {'age': 22, 'dept': 'ECE', 'marks': 88}
```

- **Iterating over nested dictionaries:**

```
for name, info in students.items():
    print(f'{name}: {info["dept"]}')

# Output: Alice: CS, Bob: IT, Charlie: CS
```

Summary

Operation	Syntax / Method	Result
Create	{'a': 1} / dict()	Dictionary
Access	d['key'] / d.get('key')	Value
Add/Modify	d['key'] = value	Updated dict
Remove (return value)	d.pop('key')	Value removed
Remove last item	d.popitem()	(key, value)
Delete key	del d['key']	Updated dict
Clear all	d.clear()	Empty dict
Get keys	d.keys()	dict_keys view
Get values	d.values()	dict_values view
Get items	d.items()	dict_items view

Summary

Update/merge	d.update(other) / d |= other	Updated dict
Set default	d.setdefault(k, v)	Value (old or new)
Membership (keys)	'key' in d	True / False
Length	len(d)	Number of pairs
Copy	d.copy()	Shallow copy
From keys	dict.fromkeys(keys, val)	New dict

Summary

- **Dictionaries store key-value pairs and are mutable.**
- **Keys must be unique and hashable; values can be any type.**
- **Core operations: accessing (d['key'], d.get()), adding, modifying,** `removing (pop, popitem, del, clear), and merging (update, |).`
- **Views: keys(), values(), items() provide dynamic reflections.**
- **Dictionary comprehensions offer a concise way to create dicts.**
- **Nested dictionaries are powerful for hierarchical data structures.**
- **Remember: 'in' checks keys, not values; use get() for safe access;** `never use mutable objects as keys.`